

IIT Jammu Summer School 2026

Course Name: IoT and Drone Building

IoT Assignment 3

Name: Mishthi Chopra

Roll no.: IoT025

Submission date: 8 July 2026

Submitted to: Mr. Majid Majeed

Repository Link:

[**https://github.com/miishthii/iot-summer-school-26**](https://github.com/miishthii/iot-summer-school-26)

Q35. REST API – Weather Data Fetcher

Program ESP32 to fetch real weather data from OpenWeatherMap API (free tier). Display on Serial Monitor: City Name, Temperature (API value vs DHT11 local value), Humidity, Weather Description. Compare API data with your local DHT11 reading and print the difference. Store your API key in a separate config.h file (do NOT commit this file – add to .gitignore).

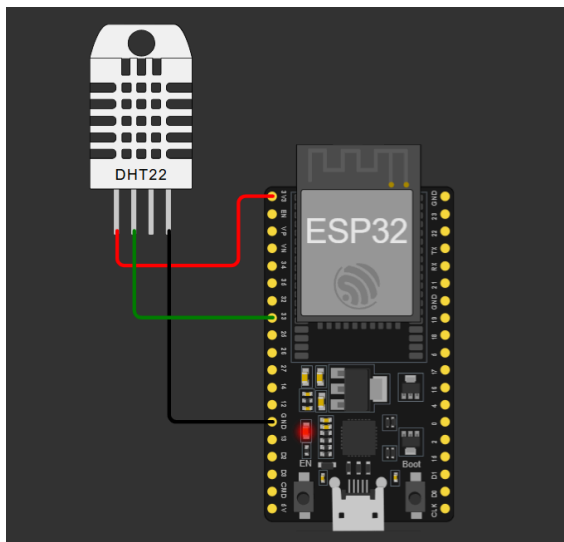
Github link:

<https://github.com/miishthii/iot-summer-school-26/tree/main/Assignments/IoTAssignment3/Q35WeatherDataFetcher>

Project link:

<https://wokwi.com/projects/468690870093711361>

Images:



WOKWI

SAVE SHARE Docs

libraries.txt • diagram.json • Library Manager • config.h •

```
1 #include <Wifi.h>
2 #include <HTTPClient.h>
3 #include <ArduinoJson.h>
4 #include <DHT.h>
5 #include "config.h"
6
7 #define DHTPIN 33
8 #define DHTTYPE DHT22
9
10 DHT dht(DHTPIN, DHTTYPE);
11
12 unsigned long previousMillis = 0;
13 const long interval = 10000; // Fetch every 10 seconds
14
15 void connectWifi()
16 {
17   Wifi.begin(WIFI_SSID, WIFI_PASSWORD);
18   Serial.print("Connecting to Wifi");
19   while (Wifi.status() != WL_CONNECTED)
20   {
21     delay(500);
22     Serial.print(".");
23   }
24   Serial.println("\nWifi Connected!");
25 }
```

Simulation

01:07.913 92%

API Temperature: 40.65 °C
Local DHT11 Temperature: 24.00 °C
Temperature Difference: -16.65 °C

API Humidity: 29 %
Local DHT11 Humidity: 40.00 %
Humidity Difference: 11.00 %

City: Jammu
Weather: broken clouds

API Temperature: 40.65 °C
Local DHT11 Temperature: 24.00 °C
Temperature Difference: -16.65 °C

API Humidity: 29 %
Local DHT11 Humidity: 40.00 %
Humidity Difference: 11.00 %

Q36. MQTT vs HTTP Comparison

Create a table comparing MQTT and HTTP across: Architecture pattern, Data transfer model, Power consumption, Latency, Use case suitability for IoT, and Security considerations. Based on your analysis, which would you choose for a 1000-node smart agriculture deployment and why?

<u>Feature</u>	<u>MQTT</u>	<u>HTTP</u>
Architecture Pattern	Publish/Subscribe using a broker	Client–Server (Request–Response)
Data Transfer Model	Devices publish messages to topics; subscribers receive them	Client sends a request, server returns a response
Power Consumption	Low power consumption due to lightweight communication	Higher power consumption because every request establishes communication overhead
Latency	Low latency, suitable for real-time communication	Higher latency due to request-response cycle
IoT Use Case Suitability	Excellent for large-scale IoT networks, sensor monitoring, and real-time data	Better for web applications, REST APIs, configuration, and occasional data requests
Security Considerations	Supports TLS/SSL encryption, authentication, and broker-based access control	Supports HTTPS (TLS/SSL), authentication, and standard web security mechanisms

I would choose **MQTT** for a 1000-node smart agriculture deployment.

Reason:

- MQTT is lightweight and consumes less bandwidth and power, making it ideal for battery-powered sensor nodes.
- It provides low-latency communication, allowing real-time monitoring of environmental parameters such as soil moisture, temperature, and humidity.
- The publish/subscribe model enables a single broker to efficiently distribute data to multiple subscribers without each sensor communicating directly with every application.
- MQTT scales well to thousands of connected devices while reducing network traffic and server load.

- Secure communication can be achieved using TLS/SSL encryption and authenticated broker connections.

Therefore, MQTT is the preferred protocol for large-scale IoT deployments like smart agriculture because it is more scalable, energy-efficient, and optimized for continuous sensor communication.

Q37. Wi-Fi Security for IoT

Research and write 250 words on: Common Wi-Fi attack vectors on IoT devices (WPA2 vulnerabilities, evil twin attacks), Best practices for securing ESP32 Wi-Fi connections (certificate pinning, WPA3), and Why hardcoding passwords in IoT source code is dangerous. Reference at least 2 sources in your answer.

Wi-Fi-enabled IoT devices are vulnerable to several security threats if proper protection is not implemented. One common attack vector is the **KRACK (Key Reinstallation Attack)**, which exploits weaknesses in the WPA2 four-way handshake to manipulate encryption keys and potentially allow attackers to intercept or modify network traffic. Another common attack is the **Evil Twin attack**, where an attacker creates a fake Wi-Fi access point with the same name as a legitimate network. IoT devices may unknowingly connect to the rogue network, allowing attackers to capture sensitive information. Devices are also vulnerable to **deauthentication attacks**, where attackers repeatedly disconnect devices from the network, affecting communication and availability.

To improve the security of **ESP32** Wi-Fi connections, developers should use **WPA3** whenever it is supported, as it offers stronger authentication and better protection against password-based attacks than WPA2. Using **TLS/SSL encryption** with **certificate pinning** helps ensure that the ESP32 communicates only with trusted servers, reducing the risk of man-in-the-middle attacks. Additional security measures include keeping firmware updated, using strong and unique Wi-Fi passwords, and isolating IoT devices on a separate network or VLAN.

Hardcoding Wi-Fi passwords or API keys directly into IoT source code is dangerous because anyone who gains access to the source code or firmware can extract these credentials. If the code is uploaded to a public repository, attackers may gain unauthorized access to the network or cloud services. A safer approach is to store credentials in a separate configuration file or using secure credential storage mechanisms.

Sources:

- <https://www.atlantis-press.com/article/126022699.pdf>
- <https://www.hologram.io/blog/krack-iot-how-the-latest-widespread-wifi-wpa2-vulnerability-is-affecting-the-internet-of-things/>
- <https://www.dspvalley.com/how-to-secure-your-iot-devices/>
- <https://arxiv.org/pdf/2601.02376>

Q38. LoRa & LPWAN – Long Range IoT

LoRa (Long Range) is widely used for IoT in agriculture and smart cities. Write 200 words explaining: What LPWAN means, How LoRa achieves long range at low power (spreading factor concept), LoRa vs Zigbee vs Z-Wave comparison, and One real-world deployment example in India.

LPWAN (Low Power Wide Area Network) is a wireless communication technology designed for IoT applications that require long-distance communication while consuming very little power. It enables battery-powered devices to transmit small amounts of data over several kilometers, making it ideal for applications such as agriculture, smart cities, and environmental monitoring.

LoRa (Long Range) is one of the most popular LPWAN technologies. It achieves long-range communication using **Chirp Spread Spectrum (CSS)** modulation and a configurable **Spreading Factor (SF)**. The spreading factor determines how long each symbol is transmitted. A higher spreading factor increases communication range and improves resistance to interference but reduces data transmission speed. This trade-off allows LoRa devices to communicate over distances of up to 15 km in rural areas while maintaining very low power consumption.

<u>Feature</u>	<u>LoRa</u>	<u>Zigbee</u>	<u>Z-Wave</u>
Range	2–15 km	10–100 m	30–100 m
Power Consumption	Very Low	Low	Low
Data Rate	Low	Medium	Low
Best Use Case	Agriculture, Smart Cities	Home Automation	Smart Homes

A notable deployment in India is **Tata Communications' LoRaWAN network**, which has been used in several cities for smart metering, environmental monitoring, and smart city infrastructure. LoRa is also increasingly used in Indian agriculture for remote monitoring of soil moisture, irrigation systems, and weather conditions, helping farmers improve water management and crop productivity.

Q39. (Theory) What is an IoT gateway? Draw a simple diagram showing the relationship between: IoT Sensor Nodes → Gateway → Cloud → User Application. What protocols are typically used at each layer? (2 marks)

An **IoT gateway** is a device that acts as an intermediary between IoT sensor nodes and the cloud. It collects data from multiple sensors, performs tasks such as data filtering, protocol conversion, and security, and then forwards the processed data to cloud servers. IoT gateways reduce network traffic, improve reliability, and allow communication between devices using different protocols.

Diagram



Protocols Used at Each Layer

Layer

Typical Protocols

**IoT Sensor Nodes →
Gateway**

Zigbee, LoRa, Bluetooth Low Energy (BLE), Wi-Fi

Gateway → Cloud

MQTT, HTTP/HTTPS, CoAP

Cloud → User Application

HTTPS, REST API, WebSocket

Q40. (Theory) Explain QoS (Quality of Service) levels in MQTT: QoS 0 (At most once), QoS 1 (At least once), QoS 2 (Exactly once). In an IoT water level monitoring system, which QoS level would you choose and why? (2 marks)

Quality of Service (QoS) in MQTT defines the reliability of message delivery between the publisher and the subscriber. MQTT provides three QoS levels:

- **QoS 0 – At most once:** The message is sent only once without confirmation. It is the fastest mode but messages may be lost if transmission fails.
- **QoS 1 – At least once:** The sender waits for an acknowledgment from the receiver. If no acknowledgment is received, the message is retransmitted. This ensures delivery but may result in duplicate messages.

- **QoS 2 – Exactly once:** The highest reliability level. It uses a four-step handshake to ensure each message is delivered exactly once without duplication, but it has the highest communication overhead.

I would choose **QoS 1 (At least once)** because reliable delivery of water level data is important to prevent missing critical alerts. While duplicate messages may occasionally occur, they can be handled by the application more easily than missing important data. QoS 1 provides a good balance between reliability, network bandwidth, and power consumption, making it suitable for most IoT monitoring applications.

Q41. Smart Home Air Quality Monitor

Problem Statement

A family in a Milan(Europe) home is concerned about indoor air quality during winter when they burn wood for heating. Design an IoT-based Air Quality Monitoring System that continuously measures CO/smoke levels (MQ-2 sensor), temperature and humidity (DHT11), and alerts residents when air quality drops below safe thresholds. The system must work without the internet when the router is down.

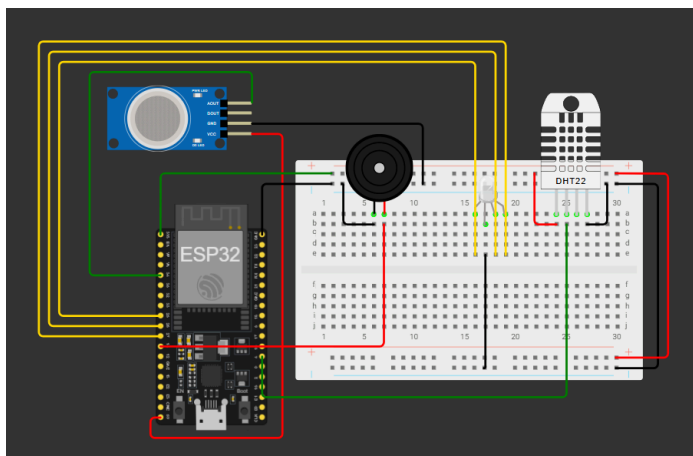
Github Link:

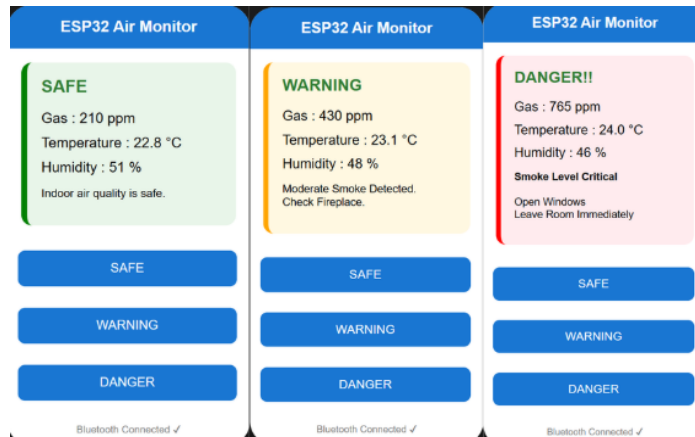
https://github.com/miishthii/iot-summer-school-26/tree/main/Assignments/loTAssignment3/Q41_AQIMonitor

Video: <https://www.loom.com/share/4d0ca5a752c14d6fb9431ef02fc5987a>

Project Link: <https://wokwi.com/projects/468812752383947777>

Images:





Q42. Automated Plant Watering System

Problem Statement

A J&K horticulture farmer wants to automate irrigation for a small saffron farm plot. The system should read soil moisture every 30 minutes and activate a water pump only when the soil is dry. Saffron is sensitive to overwatering — the system must also prevent watering if it rained recently (use LDR to detect low light/overcast sky as a rain proxy and also leverage the use of api also). The farmer must be able to manually override via a button.

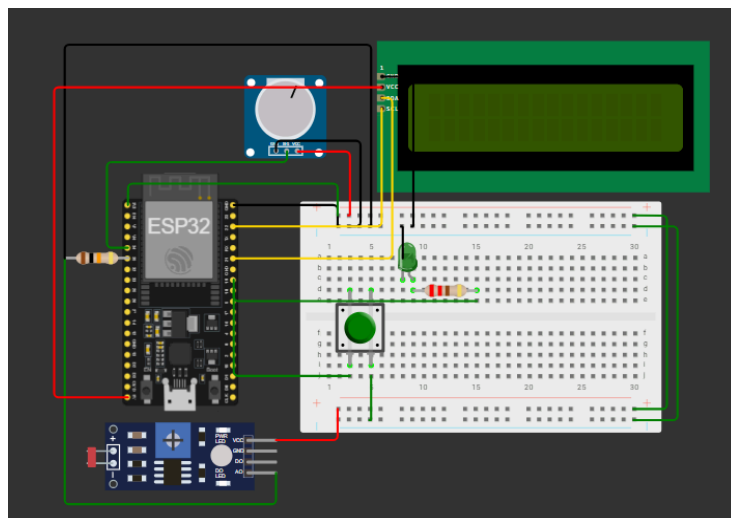
Github Link:

<https://github.com/miishthii/iot-summer-school-26/tree/main/Assignments/IoTAssignment3/Q42PlantWatering>

Video: <https://www.loom.com/share/de1deaf41923426fbde2ba8e11cd67a0>

Project Link: <https://wokwi.com/projects/468815812695429121>

Image:



Q43. Smart Door Lock with OTP

Problem Statement

A university hostel wants to replace physical keys with a PIN-based entry system that also generates a One-Time Password. Design a system where a student enters their 4-digit ID on a keypad, the system generates a 6-digit OTP, sends it over Bluetooth to their phone, and only grants access if they enter the OTP within 30 seconds.

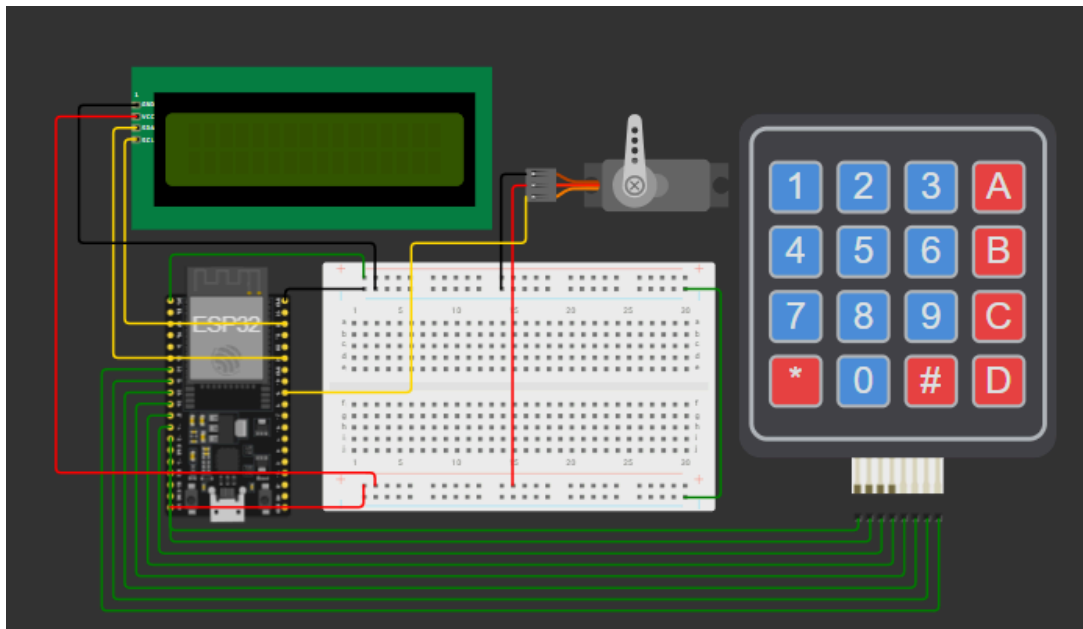
Github Link:

https://github.com/miishthii/iot-summer-school-26/tree/main/Assignments/loTAssignment3/Q43_SmartDoorLock

Video: <https://www.loom.com/share/d9fdf44abeca4de6b5fbf55a15d38995>

Project Link: <https://wokwi.com/projects/468818991301918721>

Image:



Q44. PIR-Based Security Camera Trigger

Problem Statement

A small shop in Haryana, Rohtak wants a low-cost security alert system. When motion is detected after shop hours (manually configured time window), the system should sound an alarm, flash lights, and optionally send a notification. Build the motion-detection and alerting module.

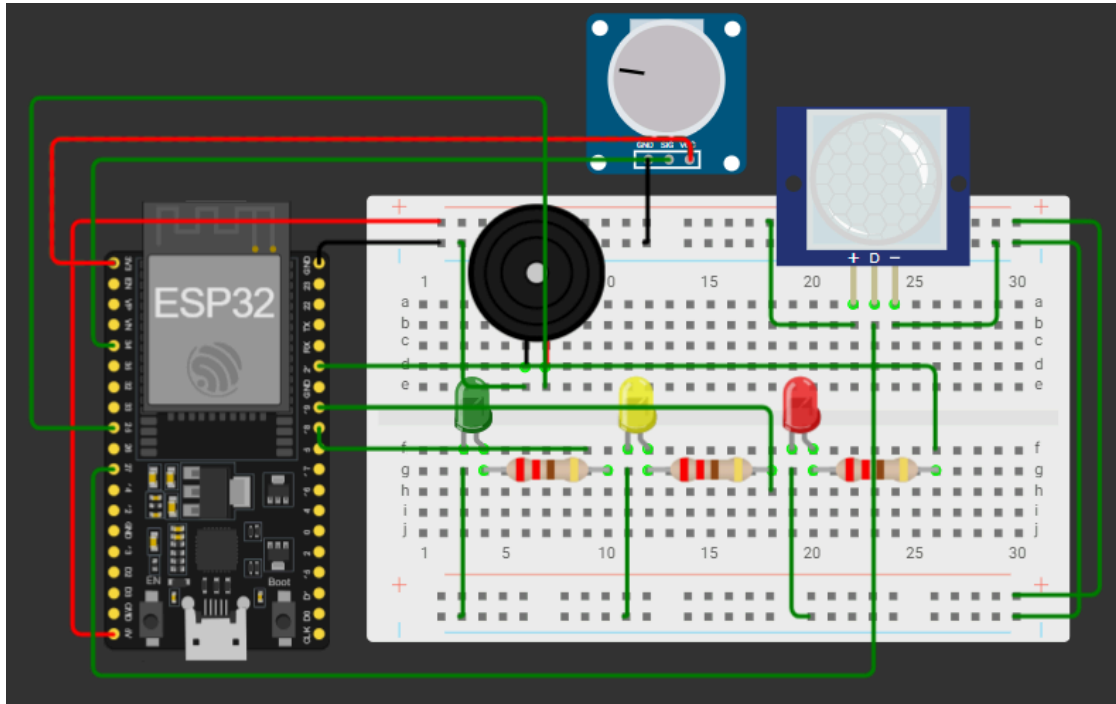
Github Link:

https://github.com/miishthii/iot-summer-school-26/tree/main/Assignments/loTAssignment3/Q44_PIRSecurity

Video: <https://www.loom.com/share/d9fdf44abeca4de6b5fbf55a15d38995>

Project Link: <https://wokwi.com/projects/468818991301918721>

Image:



Q45. Smart Greenhouse Controller

Problem Statement

Floriculture farmers in Kashmir growing tulips and roses need automated greenhouse climate control. Design a multi-parameter monitoring and control system that maintains optimal growing conditions: Temperature 18–22°C, Humidity 60–70%, and sufficient light for 12 hours per day.

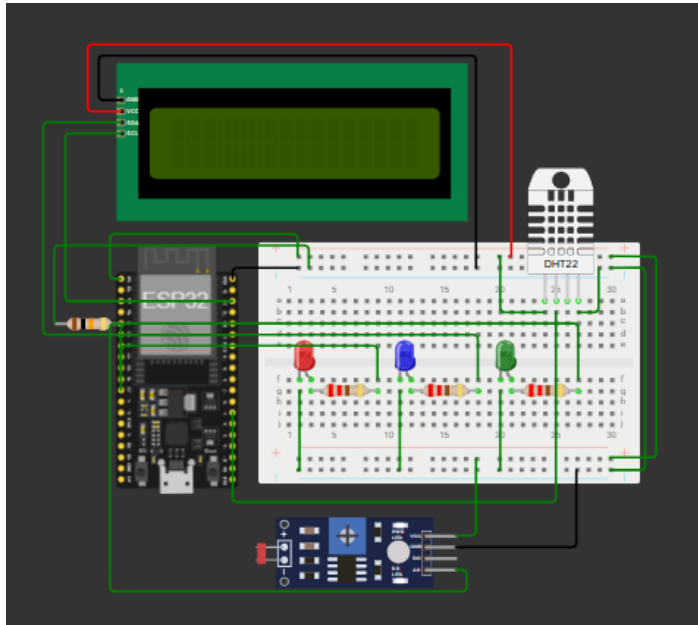
Github Link:

https://github.com/miishthii/iot-summer-school-26/tree/main/Assignments/loTAssignment3/Q45_GreenhouseController

Video: <https://www.loom.com/share/8fd85ec5f68b4695809637a52efec972>

Project Link: <https://wokwi.com/projects/468860043115747329>

Images:



Websockets Client Showcase



Need a fully managed MQTT broker?
Get your own Cloud broker and connect up to 100 devices for free.

[Get your free account](#)

Connection

● connected

Publish

Messages

2028-07-07 10:11:05	Topic: greenhouse/temp	Qos: 0
75.90		
2028-07-07 10:11:03	Topic: greenhouse/fan	Qos: 0
1		
2028-07-07 10:11:03	Topic: greenhouse/lightStatus	Qos: 0
1		
2028-07-07 10:11:03	Topic: greenhouse/humidity	Qos: 0
81.00		
2028-07-07 10:11:03	Topic: greenhouse/heater	Qos: 0
0		
2028-07-07 10:11:03	Topic: greenhouse/light	Qos: 0
57		
2028-07-07 10:11:02	Topic: greenhouse/temp	Qos: 0
75.90		
2028-07-07 10:11:00	Topic: greenhouse/lightStatus	Qos: 0
1		
2028-07-07 10:11:00	Topic: greenhouse/fan	Qos: 0
1		
2028-07-07 10:11:00	Topic: greenhouse/humidity	Qos: 0
81.00		
2028-07-07 10:11:00	Topic: greenhouse/heater	Qos: 0
1		
2028-07-07 10:11:00	Topic: greenhouse/light	Qos: 0
57		
2028-07-07 10:10:59	Topic: greenhouse/temp	Qos: 0
-33.10		
2028-07-07 10:10:57	Topic: greenhouse/fan	Qos: 0
1		
2028-07-07 10:10:57	Topic: greenhouse/lightStatus	Qos: 0
1		
2028-07-07 10:10:57	Topic: greenhouse/humidity	Qos: 0
81.00		

Subscriptions

Add New Topic Subscription

Qos: 2	greenhouse/temp	X
Qos: 2	greenhouse/humidity	X
Qos: 2	greenhouse/light	X
Qos: 2	greenhouse/heater	X
Qos: 2	greenhouse/fan	X
Qos: 2	greenhouse/lightStatus	X

Q46. River Water Level Alert (Flood Detection)

Problem Statement

Given the frequent flooding near the Tawi and Chenab rivers, design a low-cost IoT flood-early-warning sensor node using an HC-SR04 ultrasonic sensor to monitor water level in a model river (a glass tank). Trigger alerts at predefined levels and transmit data wirelessly.

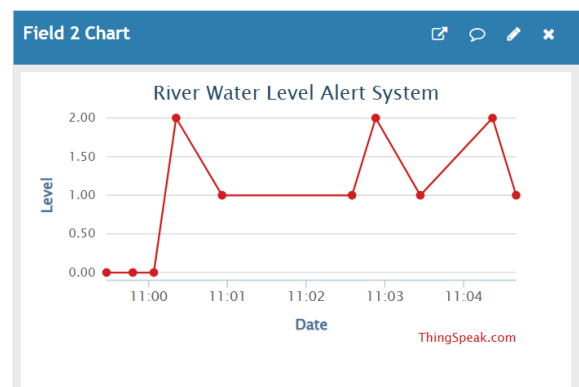
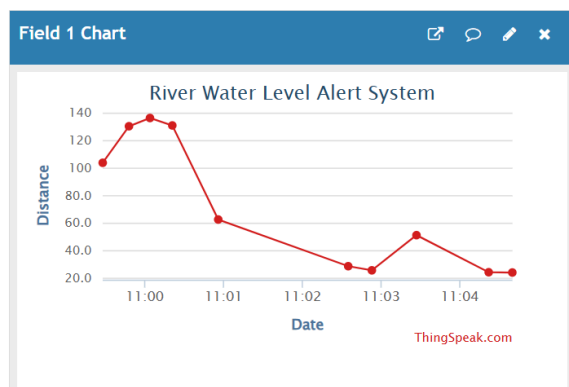
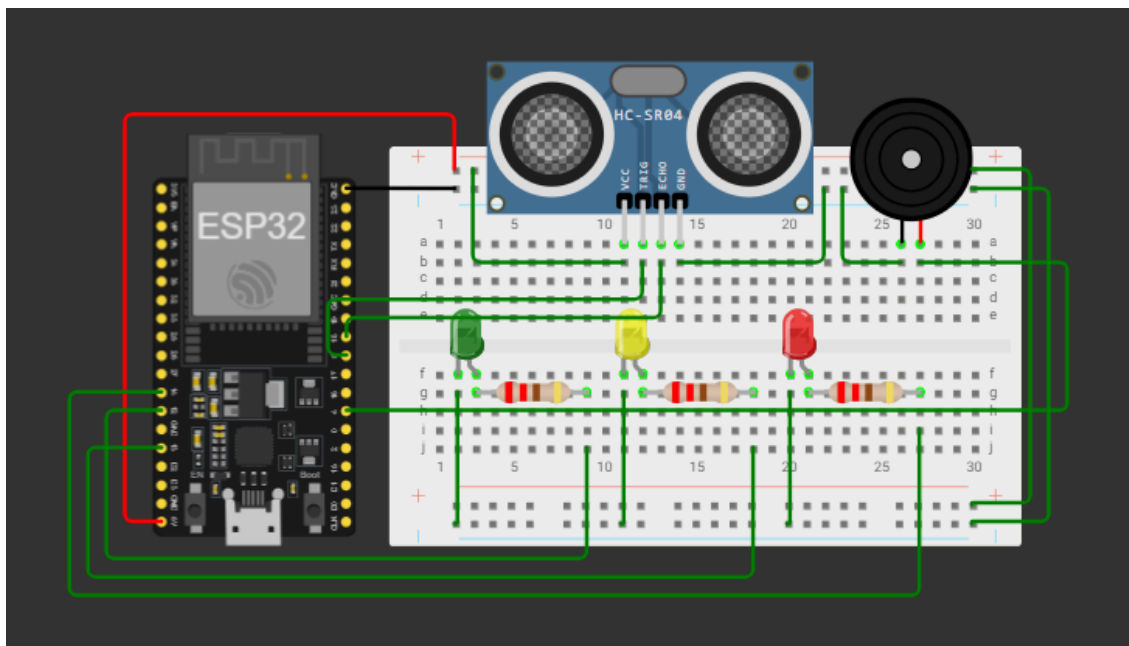
Github Link:

https://github.com/miishthii/iot-summer-school-26/tree/main/Assignments/loTAssignment3/Q46_WaterLevel

Video: <https://www.loom.com/share/eeb74a4069444ec5a15c795efac85a52>

Project Link: <https://wokwi.com/projects/468863638831329281>

Images:



Q47. Heart Rate & SpO2 Monitor (Simulation)

Problem Statement

Using a MAX30102 pulse oximeter module (or simulate with an LDR as a proxy sensor), build a health monitoring prototype that measures pulse rate and blood oxygen saturation, displays values, and alerts on abnormal readings. This is a simulation for learning sensor interfacing, not a medical device.

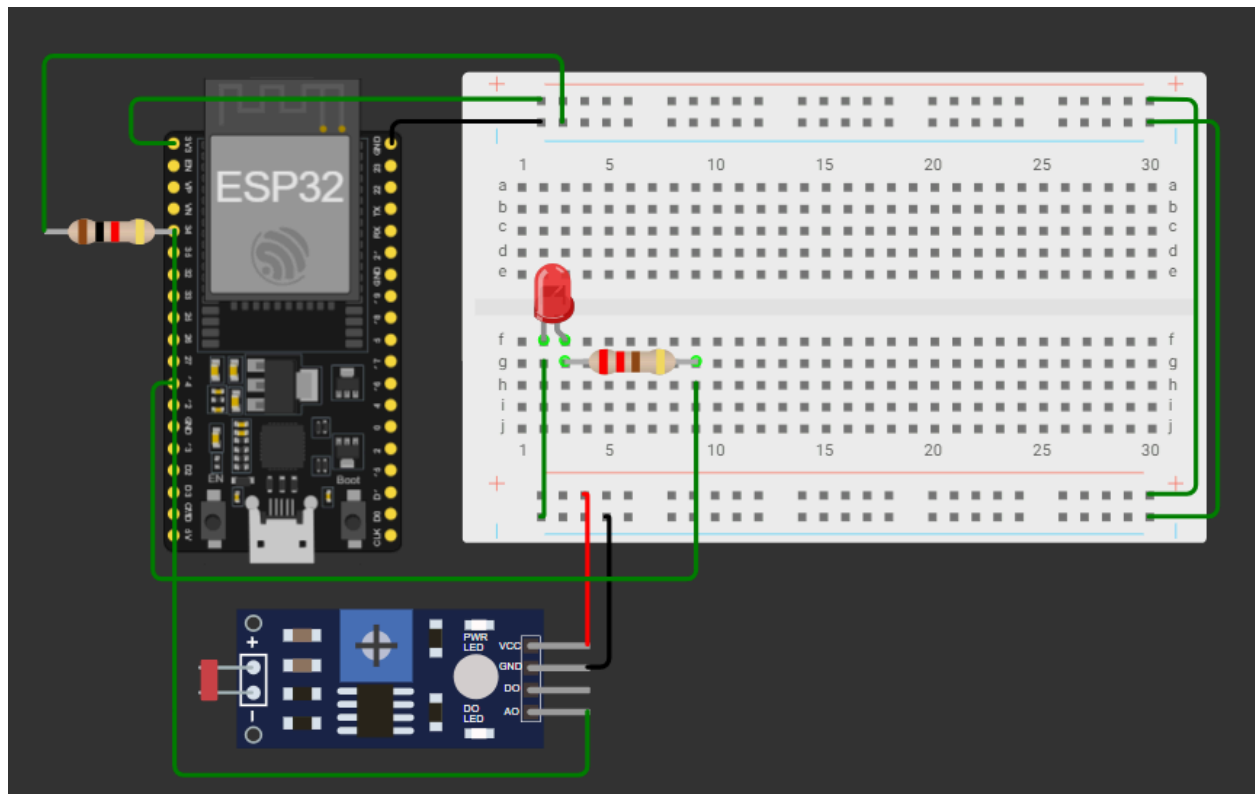
Github Link:

https://github.com/miishthii/iot-summer-school-26/tree/main/Assignments/loTAssignment3/Q47_HeartRateMonitor

Video: <https://www.loom.com/share/96358ea41e4d4aa1ae6f9188194ef12a>

Project Link: <https://wokwi.com/projects/468865237422386177>

Images:



Q48. Design Challenge: IoT for Jammu Smart City

J&K Government is planning a Smart City project for Jammu. As an IoT engineer, propose a system to monitor and manage 3 of the following (choose any 3): Street lighting, Traffic flow, Parking availability, Air quality, Water supply pressure, Waste bin fill levels. For each:

- Identify the sensor(s) needed
- Identify the microcontroller / connectivity module

- Describe the data flow: Sensor → Protocol → Cloud → Dashboard
- Estimate approximate cost per node (in INR)

Present your answer as a structured proposal (500–700 words) uploaded as /final_project/smart_city_proposal.md in GitHub.

1. Write the code for each problem separately.

Github Link:

https://github.com/miishthii/iot-summer-school-26/tree/main/Assignments/loTAssignment3/Q48_JammuSmartCity

Video:

Smart Street Light: <https://www.loom.com/share/21b751d46f1e4e28a70f3bf6fcc95b24>

AQI Monitor: <https://www.loom.com/share/2c990c8136db4413a660aaac324263e7>

Smart Wastebin: <https://www.loom.com/share/5a1c7d995d6f4fe99641c0e3c9d6fad9>

Project Link:

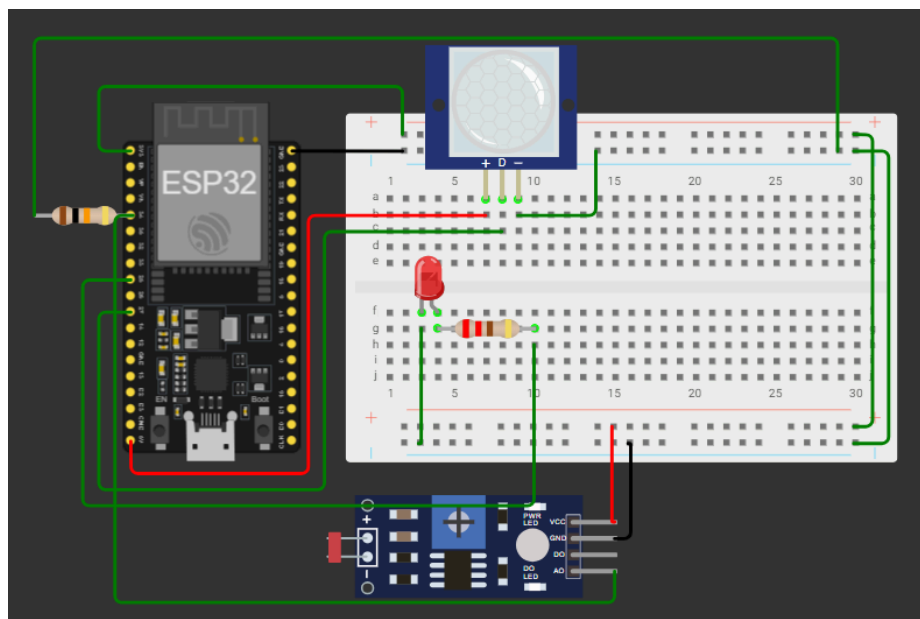
Smart Street Light: <https://wokwi.com/projects/468906013313278977>

AQI Monitor: <https://wokwi.com/projects/468907101773073409>

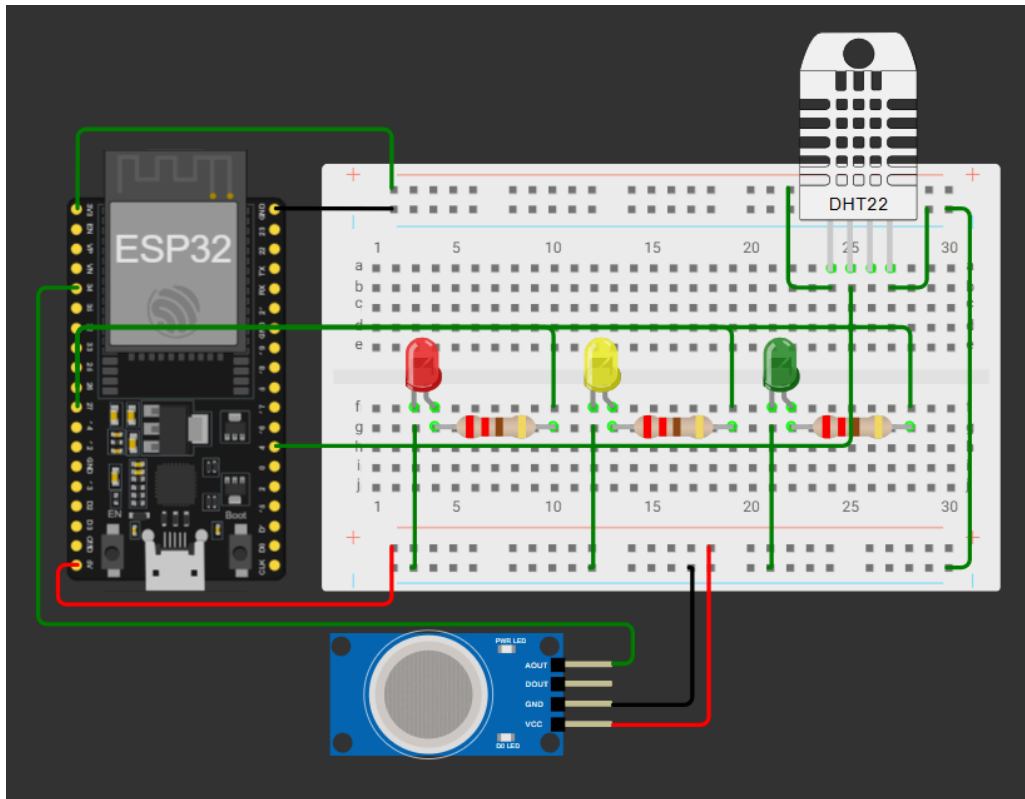
Smart Wastebin: <https://wokwi.com/projects/468907991287560193>

Images:

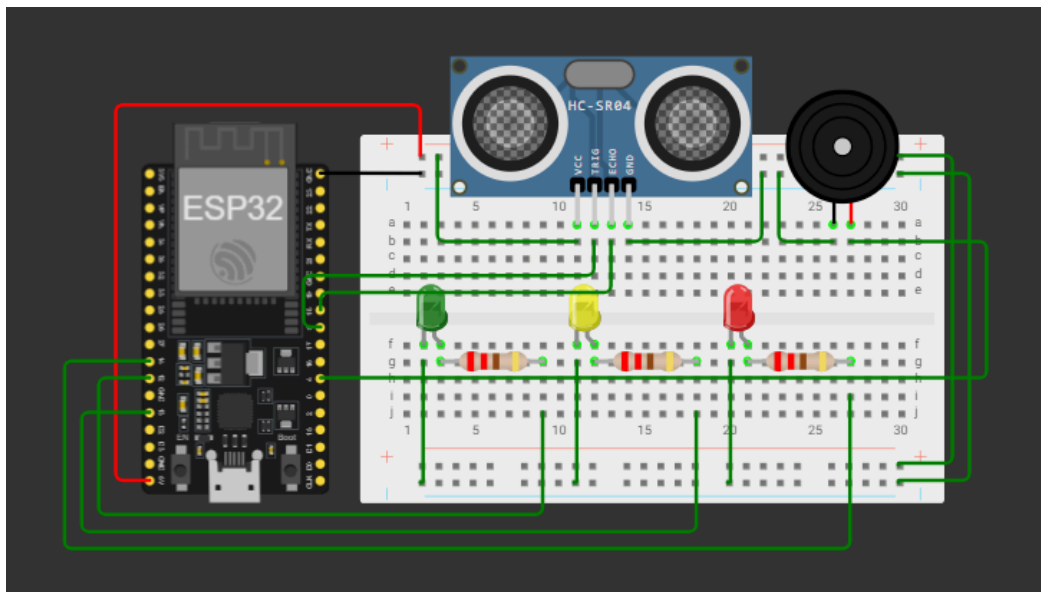
Smart Street Light:



AQI Monitor:



Smart Wastebin:



Q49. Security Vulnerability Analysis

You are given a poorly written IoT code snippet that connects an ESP8266 to a public Wi-Fi network. The code hardcodes the SSID and password, uses unencrypted HTTP (not HTTPS), publishes sensor data to a public MQTT broker without authentication, and never validates incoming commands. Identify at least 5 security vulnerabilities, explain the risk of each, and provide corrected code for at least 2 vulnerabilities.

Security Vulnerabilities

<u>Vulnerability</u>	<u>Risk</u>	<u>Recommended Solution</u>
Hardcoded Wi-Fi SSID and Password	Credentials can be extracted from the source code or firmware, allowing attackers to gain unauthorized access to the network.	Store credentials in a separate <code>config.h</code> file excluded using <code>.gitignore</code> , or use secure credential storage.
Using HTTP Instead of HTTPS	Data is transmitted without encryption, making it vulnerable to eavesdropping and man-in-the-middle (MITM) attacks.	Use HTTPS with TLS/SSL encryption and validate server certificates.
Public MQTT Broker Without Authentication	Anyone can publish or subscribe to topics, allowing unauthorized access, spoofing, or data tampering.	Use an authenticated MQTT broker with username/password and TLS encryption.
No Validation of Incoming Commands	Attackers can send malicious or unexpected commands, potentially causing unsafe device behavior.	Validate all incoming commands and allow only predefined, authorized commands.
No Access Control or Authentication	The device cannot verify the identity of clients, increasing the risk of unauthorized control.	Implement authentication mechanisms such as API keys, tokens, or client certificates.

Corrected Code Example 1 – Secure Wi-Fi Credentials

Before:

```
const char* ssid = "PublicWiFi";  
const char* password = "12345678";
```


After:

```
#include "config.h"
WiFi.begin(WIFI_SSID, WIFI_PASSWORD);
```

config.h

```
#ifndef CONFIG_H
#define CONFIG_H

const char* WIFI_SSID = "YOUR_WIFI_NAME";
const char* WIFI_PASSWORD = "YOUR_WIFI_PASSWORD";

#endif
```

Corrected Code Example 2 – Use HTTPS Instead of HTTP**Before:**

```
HTTPClient http;
http.begin("http://example.com/api");
```

After:

```
WiFiClientSecure client;
client.setInsecure(); // For testing only

HTTPClient https;
https.begin(client, "https://example.com/api");
```

Conclusion

Securing IoT devices requires protecting sensitive credentials, encrypting network communication, authenticating users and devices, and validating all incoming data. Following these security best practices significantly reduces the risk of unauthorized access, data theft, and malicious attacks.

Q50. Reflection & Learning Portfolio

Describe what you built and what you learned. How will you apply IoT skills in your field of study or future career? What would you build next — describe your own IoT project idea with sensors, connectivity, and application. (Minimum 400 words, personal and reflective)

During this IoT Summer School, I gained hands-on experience in designing, programming, and documenting Internet of Things (IoT) systems using ESP32 and Arduino-based platforms. At the beginning of the course, I had only a basic understanding of embedded systems, but through practical assignments and simulations, I learned how sensors, microcontrollers, communication protocols, and cloud services work together to build complete IoT solutions.

I worked on projects involving temperature and humidity monitoring with the DHT11 sensor, REST API integration using OpenWeatherMap, MQTT communication, HTTP requests, and Bluetooth-based communication. I also learned how to interface different sensors such as gas sensors, ultrasonic sensors, LDRs, soil moisture sensors, and PIR motion sensors. Along the way, I became familiar with Wokwi simulation, circuit design, GitHub repositories, README documentation, and organizing projects using proper software engineering practices. Understanding concepts like Wi-Fi security, Quality of Service (QoS), LPWAN technologies such as LoRa, and IoT gateways helped me appreciate the networking side of IoT in addition to hardware programming.

As an Electronics and Communication Engineering (ECE) student, these skills have strengthened my understanding of embedded systems and real-world automation. I now understand how sensor data can be collected, processed, transmitted, and visualized to solve practical problems in areas such as smart homes, agriculture, healthcare, and environmental monitoring. The course also improved my problem-solving skills, debugging techniques, and ability to integrate multiple hardware and software components into a single project.

In my future career, I hope to apply these IoT skills in embedded systems development and industrial automation. I am particularly interested in designing intelligent devices that combine sensors, wireless communication, and cloud technologies to improve efficiency and safety. Learning protocols such as HTTP, MQTT, Bluetooth, and Wi-Fi has given me a strong foundation for developing connected embedded systems.

If I were to build my own IoT project next, I would further develop my **Smart Room Comfort System** into a more advanced smart home solution. The system would use **DHT11** sensors to monitor temperature and humidity, an **LDR** to measure ambient light levels, and weather data from the **OpenWeatherMap API** to provide both indoor and outdoor environmental information. An **ESP32** would collect and process the sensor data while communicating with a cloud dashboard using **MQTT**. Based on the collected data, the system could automatically control appliances such as fans, lights, or air conditioners to maintain a comfortable indoor environment while reducing energy consumption. Users could monitor room conditions in real time through a web or mobile dashboard and receive alerts whenever the temperature, humidity, or weather

conditions exceed predefined thresholds. This project combines environmental monitoring, cloud connectivity, automation, and energy efficiency, making it suitable for smart homes, offices, classrooms, and hostels.

Overall, this IoT Summer School has provided me with practical experience in embedded systems, sensor interfacing, wireless communication, cloud connectivity, and project documentation. It has increased my confidence in designing complete IoT solutions and motivated me to continue exploring advanced embedded systems, edge computing, and smart automation technologies in the future.